



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 09

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 19 de abril del 2026

CALIFICACIÓN: _____

ACTIVIDADES REALIZADAS

EJERCICIO 1. Coche

Instrucciones: “Hacer que el carro avance 50 unidades, de vuelta de 180 grados y regrese al punto de partida, las llantas deben de ir girando mientras el carro avanza. La animación es continua o cíclica.”

En este ejercicio no se tuvo que modificar nada en los archivos window.h o window.cpp más allá del nombre de ventana, es por ello que todas las modificaciones importantes se realizaron dentro del main.

MAIN

Ajustamos primeramente la velocidad de avance del coche y la velocidad de giro de las llantas para apreciar mejor la animación, para después, dentro de nuestro bucle while por cada ciclo de reloj, modificar la lógica de la animación ping-pong. Esta nueva lógica nos ayuda asegurar el avance de las 50 unidades cíclicas al verificar con nuestro if/else mediante la bandera avanza si el coche va hacia lejos del centro (0,0,0) o acercándose a este.

```
movCoche = 0.0f;
movOffset = 0.1f;           // Ajusta la velocidad de avance
rotllanta = 0.0f;
rotllantaOffset = 2.0f;     // Ajusta la velocidad de giro de llantas
...
// 1. Lógica Ping-Pong del Coche (Movimiento en X) -----
rotllanta += rotllantaOffset * deltaTime; // Rotación continua de las llantas
if (avanza) {
    // Si avanza, RESTAMOS para que viaje hacia la dirección correcta en X
    movCoche -= movOffset * deltaTime;
    if (movCoche <= -50.0f) { // Límite de 50 unidades (hacia los negativos)
        movCoche = -50.0f;   // Tope exacto
        avanza = false;      // Da la vuelta
    }
}
else {
    // Si retrocede, SUMAMOS para que regrese a 0
    movCoche += movOffset * deltaTime;
    if (movCoche >= 0.0f) { // Rregresamos al punto de partida
        movCoche = 0.0f;    // Tope exacto
        avanza = true;      // Vuelve a avanzar dando la vuelta
    }
}
```

Y es precisamente con la bandera avanza que se verifica si nuestro coche debe aplicar una rotación en su carrocería o no.

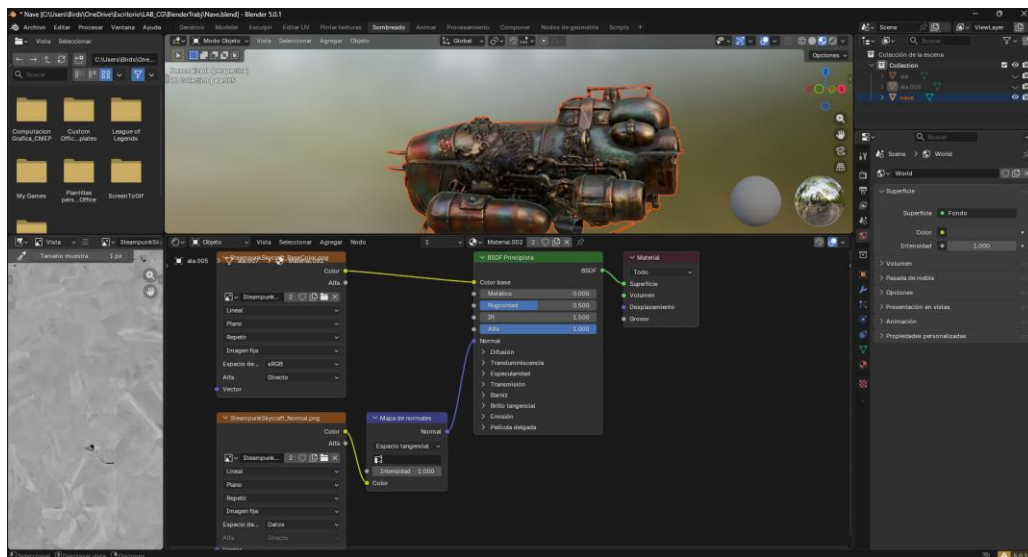
```
// COCHE Y CARROCERÍA -----
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(movCoche, -1.0f, 0.0f));
if (!avanza) model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
CocheT.RenderModel();
```

EJERCICIO 2. Nave

Instrucciones: “Hacer que la nave avance 20 unidades hacia adelante, pero subiendo y bajando $[-2,2]$ unidades en y, de giro de 180 grados y regrese al punto de partida, el ala separada debe de ir moviéndose dando la apariencia de que "aletea". La animación es continua o cíclica.”

En este ejercicio no se tuvo que modificar nada en los archivos window.h o window.cpp más allá del nombre de ventana, es por ello que todas las modificaciones importantes se realizaron dentro del main, sin embargo, comenzamos haciendo los cambios importantes en Blender.

Se decidió separar ambas alas de modelo base, por lo que se exporto la nave.obj y se quitó el ala sobrante además de agregarle los mapas de normales (en espacio de Datos) y textura (en espacio de sRGB) ya que se había escalado la imagen y se desvinculo de los archivos .mtl que se tenían dados por el profesor.



MAIN

Agregamos las variables de apoyo para la animación de la nave, seguimos el mismo tipo de variables que para coche.

```
// ===== VARIABLES PARA ANIMACIÓN =====
// --- Variables Generales (Texturas y Rotaciones de la Escena) ---
float toffsetflechau = 0.0f;
float toffsetflechav = 0.0f;
float toffsetnumerou = 0.0f;
float toffsetnumerov = 0.0f;
float toffsetnumerocambiau = 0.0f;
float angulovaria = 0.0f;
// --- Animación: Coche ---
bool avanza; // Estado de dirección (true = avanza, false = retrocede)
float movCoche; // Posición actual del coche en su eje de traslación
float movOffset; // Velocidad de avance/retroceso del coche
float rotllanta; // Ángulo actual de las llantas
float rotllantaOffset; // Velocidad a la que giran las llantas
// --- Animación: Nave (Avance en X) ---
bool avanzaNave; // Estado de dirección de la nave
float movNave; // Posición actual de avance de la nave
float movOffsetNave; // Velocidad a la que avanza la nave
// --- Animación: Nave (Zigzag en Y) ---
bool subeNave; // Estado vertical (true = sube, false = baja)
float alturaNave; // Posición actual flotante en Y
float velocidadAlturaNave; // Qué tan rápido sube o baja la nave
// --- Animación: Nave (Aleteo de alas en Z) ---
bool aleteaSube; // Estado del aleteo (true = rotando hacia arriba)
float rotAleteo; // Ángulo actual en el que se encuentra el ala
float velAleteo; // Qué tan rápido rotan las alas (grados por segundo)
float limiteAleteo; // Límite de rotación máxima (amplitud del aleteo)
```

Al igual que como hicimos con nuestro modelo de coche, exportamos los modelos de la

```
nave
LlantaT = Model(); LlantaT.LoadModel("Models/llantaT1.obj");
CocheT = Model(); CocheT.LoadModel("Models/CocheT.obj");
CofreT = Model(); CofreT.LoadModel("Models/CofreT.obj");
NaveT = Model(); NaveT.LoadModel("Models/NaveT.obj");
Ala1T = Model(); Ala1T.LoadModel("Models/Ala1T.obj");
Ala2T = Model(); Ala2T.LoadModel("Models/Ala2T.obj");
```

Sabemos que tenemos que ajustar algunas de nuestras variables declaradas al inicio y además instanciar declaraciones que no debemos poner dentro de nuestro bucle while por cada ciclo de reloj, mismas que servirán para colocar el faro de luz en la nave como se hizo con el helicóptero anteriormente.

```
// Estados iniciales para: Nave y faro -----
// --- Jerarquía de la Luz Spot ---
glm::mat4 matrizNaveLuz;
glm::vec4 posLocalFaroNave = glm::vec4(-1.0f, -0.5f, 0.0f, 1.0f); // Hacia adelante y abajo
glm::vec4 posGlobalFaroNave;
glm::vec3 dirLocalFaroNave;
glm::vec3 dirGlobalFaroNave;
// --- Avance (Ping-Pong en X) ---
avanzaNave = true;
movNave = 0.0f;
movOffsetNave = 0.1f; // Velocidad a la que avanza la nave
// --- Zigzag (Sube y baja en Y) ---
subeNave = true; // Booleano para el ping-pong vertical
alturaNave = 0.0f;
```

```

velocidadAlturaNave = 0.05f;           // Qué tan rápido sube/baja
// --- Aleteo (Rotación en Z) ---
aleteaSube = true;                     // Booleano para el ping-pong de las alas
rotAleteo = 0.0f;                      // Ángulo actual del ala
velAleteo = 0.5f;                      // Qué tan rápido rotan las alas
limiteAleteo = 30.0f;                  // Amplitud máxima del aleteo

```

Se agrego más lógica de animación, pero con más condicionales debido a las múltiples traslaciones y rotaciones, con ayuda de estos bloques if/else verificamos que se avance 20 unidades, que se suba y baje en un rango de -2 a 2, que haya una forma cíclica de esta animación y que haya una rotación para ambas alas.

```

// 2. Lógica Ping-Pong de la Nave (Avanza hacia -20 en X) -----
if (avanzaNave) {
    // Si avanza, RESTAMOS para que viaje hacia el -20
    movNave -= movOffsetNave * deltaTime;
    if (movNave <= -20.0f) {           // Llegó al límite negativo
        movNave = -20.0f;
        avanzaNave = false; // Da la vuelta para regresar
    }
}
else {
    // Si regresa, SUMAMOS para que vuelva al origen (0)
    movNave += movOffsetNave * deltaTime;
    if (movNave >= 0.0f) {             // Llegó al inicio
        movNave = 0.0f;
        avanzaNave = true;  // Vuelve a avanzar hacia el -20
    }
}

// 3. Lógica ZIGZAG de la Nave (Sube y Baja en Y) -----
if (subeNave) {
    alturaNave += velocidadAlturaNave * deltaTime;
    if (alturaNave >= 2.0f) {          // Llegó al tope superior
        alturaNave = 2.0f;
        subeNave = false; // Ahora baja
    }
}
else {
    alturaNave -= velocidadAlturaNave * deltaTime;
    if (alturaNave <= -2.0f) {         // Llegó al tope inferior
        alturaNave = -2.0f;
        subeNave = true;  // Ahora sube
    }
}

// 4. Lógica Ping-Pong del Aleteo (Rotación en Z) -----
if (aleteaSube) {
    rotAleteo += velAleteo * deltaTime;
    if (rotAleteo >= limiteAleteo) { // Llegó al límite superior de rotación
        rotAleteo = limiteAleteo;
        aleteaSube = false; // Ahora rota hacia abajo
    }
}
else {
    rotAleteo -= velAleteo * deltaTime;
    if (rotAleteo <= -limiteAleteo) { // Llegó al límite inferior de rotación
        rotAleteo = -limiteAleteo;
        aleteaSube = true;  // Ahora rota hacia arriba
    }
}

```

Finalmente se renderizaron los tres modelos donde están sujetos a las variables movNave y alturaNave para la traslación, así como evaluaciones de la bandera avanzaNave para aplicar la rotación de 180° al modelo padre.

```

// NAVE Y SUS ALAS (Jerarquía Zigzag + Aleteo) -----
model = glm::mat4(1.0);

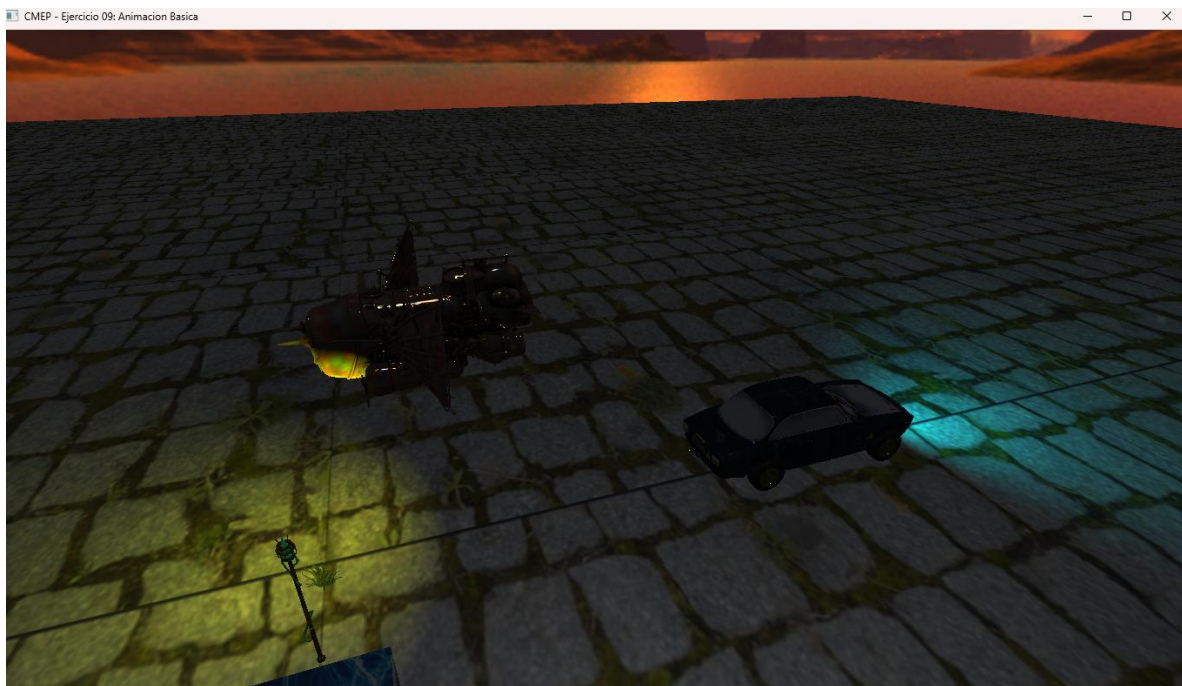
```

```

model = glm::translate(model, glm::vec3(movNave, 13.0f + alturaNave, 1.5f));
if (!avanzaNave) {
    model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
}
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
NaveT.RenderModel();
// ALA 1 (Costado Derecho)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.5f));
model = glm::rotate(model, rotAleteo * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala1T.RenderModel();
// ALA 2 (Costado Izquierdo)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.5f));
model = glm::rotate(model, -rotAleteo * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala2T.RenderModel();

```

EJECUCION



Se adjunta el link del GIF en drive.

<https://drive.google.com/file/d/1VNi5pVxt2nYqsVuecwStlkByVbxULpBJ/view?usp=sharing>

PROBLEMAS PRESENTADOS

No hubo mayor complejidad más que al exportar los modelos y verificar el avance correcto, al intentar hacer los bloque if/else el coche y nave iban en reversa en ambas direcciones, esto se solucionó identificando las variables de posición global y modificando los signos positivos y negativos de algunas rotaciones. Además, al instanciar las alas estas no solo iban desfasadas, sino que rotaban en un eje incorrecto, nuevamente la solución a estas problemáticas fue identificar bien donde estaban nuestras rotaciones y en que eje se hacían.

CONCLUSION

Sobre los ejercicios de clase

Considero que los ejercicios no tuvieron una complejidad elevada, pero si un trabajo algo pesado, el instanciar de nuevo nuestros propios modelos al código proporcionado por el profesor fue tardado, y cuando finalmente se completo esta tarea al analizar todo el código lo ideal hubiera sido hacer lo contrario, modificar el main que ya se tenia para agregar unas cuantas variables, modelos y la lógica de animación, aunque se perdió tiempo en esto, sirvió como un repaso de todo nuestro trabajo al momento. Se entendio también que pasar de animaciones lineales simples a implementar múltiples 'máquinas de estados' simultáneas (como las vistas en materias como diseño digital) demuestra la potencia de las transformaciones geométricas y el uso del deltaTime

Comentarios generales

Para esta práctica ya todo en su mayoría es entendible, aunque se hubiera preferido dar más énfasis a las variables usadas en la lógica de animación ya que su entendimiento del porque estaban ahí y como se usaban se dio hasta mediados de la realización de esta práctica.

REFERENCIAS

- Nystrom, R. (2014). State. En *Game Programming Patterns*. Consultado el 18 de abril del 2026 en <https://gameprogrammingpatterns.com/state.html>